

dummy_case_study_july_04_2026

We will use a tiny customer-support classifier with four input features:

*
*
*
*

The model produces two scores:

*
*

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
np.set_printoptions(precision=3, suppress=True)
torch.set_printoptions(precision=3, sci_mode=False)
feature_names = ["refund", "broken", "thank_you", "general_question"]
class_names = ["urgent", "normal"]
```

I want a refund, I got a broken phone.

Code

```
x = np.array([1.0, 1.0, 0.0, 0.0])
pd.DataFrame({
    "feature": feature_names,
    "value": x
})
```

Output

	feature	value
0	refund	1.0
1	broken	1.0
2	thank_you	0.0
3	general_question	0.0

Code

```
W_base = np.array([
    [0.10, 0.20, -0.10, 0.10], # urgent weights
    [0.50, 0.40, 0.30, 0.50]  # normal weights
])
base_weight_table = pd.DataFrame(
    W_base,
    index=class_names,
    columns=feature_names
)
base_weight_table
```

Output

refund	broken	thank_you	general_question	
urgent	0.1	0.2	-0.1	0.1
normal	0.5	0.4	0.3	0.5

Weve 4 features, f1, f2, f3 and f4

And these are corresponding weights for them, 16, 2, -8, 4

Code

```
base_scores = W_base @ x
base_score_table = pd.DataFrame({
    "class": class_names,
    "score": base_scores
})
base_prediction = class_names[np.argmax(base_scores)]
print(base_score_table.to_string(index=False))
print(f"\nBase prediction: {base_prediction}")
```

Output

class	score
urgent	0.3
normal	0.9

Base prediction: normal

Code

```
delta_w = np.array([
[ 0.60,  0.50, 0.00, 0.00],
[-0.18, -0.15, 0.00, 0.00]
])
delta_weight_table = pd.DataFrame(
delta_w,
index=class_names,
columns=feature_names
)
delta_weight_table
```

Output

refund	broken	thank_you	general_question	
urgent	0.60	0.50	0.0	0.0
normal	-0.18	-0.15	0.0	0.0

Code

```
w_effective = w_base + delta_w
```

Code

```
adapted_scores = W_effective @ x
adapted_prediction = class_names[np.argmax(adapted_scores)]
score_comparison = pd.DataFrame({
    "class": class_names,
    "base score": base_scores,
    "adapted score": adapted_scores
})
print(score_comparison.to_string(index=False))
print(f"\nBase prediction: {base_prediction}")
print(f"Adapted prediction: {adapted_prediction}")
```

Output

```
class  base score  adapted score
urgent         0.3         1.40
normal         0.9         0.57
Base prediction: normal
Adapted prediction: urgent
```